



INDIA.RUNS

Build what next India runs on

Team Name : Parivartan

Team Leader Name : Khushi Roongta

Problem Statement : Filter 100,000 candidate profiles to find the top 100 fits for the Senior AI Engineer (Founding Team) role. The system must isolate honeypots, run under 5 minutes on CPU with zero network calls, and generate fact-based justifications.

Solution Overview

Proposed Solution

- **Honeypot Screening:** 9 logical checks filter 25,717 synthetic profiles with 0% leakage in the top 100.
- **Disqualification Filters:** Discards service-only careers, academic-only profiles, and irrelevant job titles (38,945 profiles).
- **Hybrid Ranker:** Scores candidates on a 100-point JD-aligned matrix scaled by platform engagement signals.

Unique Factors

- **No API Dependencies:** Rule-based logical check parser runs offline in ~9 seconds (100 MB RAM).
- **Honeypot Identification:** Detects chronological inconsistencies (e.g. working at CRED before it existed) that semantic search misses.
- **Total Transparency:** The Streamlit sandbox lets recruiters audit exactly how every candidate score is calculated.

JD Understanding & Candidate Evaluation

Key requirements extracted from the JD

- **Experience Sweet-Spot:** 5–9 years (ideal for Senior Founding Engineer).
- **Applied ML Tech Stack:** Vector Search (Qdrant, Pinecone), Deep Learning (PyTorch, LoRA, PEFT), and Backend Systems (Python, Docker).
- **Mindset:** Scrappy product shippers. Pure academic researchers and consulting profiles are disqualified.
- **Location:** Noida, Pune, Delhi NCR, and major Indian tech hubs.

How does our solution evaluate fit beyond keyword matching

- **Proficiency Grading:** We evaluate usage duration and skill level (Expert/Advanced/Intermediate) instead of just keyword presence.
- **Tenure Stability:** Computes average company stay, penalizing job-hoppers (tenure <1.5 years).
- **Availability Scaling:** Factors notice period, activity recency, and open-to-work status dynamically.

Ranking Methodology

How does our system retrieve, score, and rank candidates

- Streams database line-by-line
- Screens out honeypots (Gate 1) and disqualifies academic roles (Gate 2)
- Scores valid candidates
- Sorts by Score desc, Candidate ID asc.

What models, algorithms, or heuristics are used?

- Graded heuristics score experience, title, and skills.
- Platform activity signals act as multiplicative weights.
- Deterministic ID hashing selects unique recruiter reasoning templates.

Final Score = Raw Structured Score (Max 100) x Combined Behavioral Multiplier
(Multiplicative design ensures a major red flag, like >6 months inactivity, heavily penalizes overall rank).

Explainability & Data Validation

How are ranking decisions explained

- All justification texts are generated using local Python string formatting from raw candidate data.
- Since no LLM is used, it is mathematically impossible to hallucinate or mention skills/companies not present in the candidate profile.

How does our solution handle low-quality or suspicious profiles?

- Filters out profiles matching 9 logical contradictions (e.g., claiming expert skill with 0 duration, starting first job before college ended, or having salary min > max).

How do we prevent hallucinations

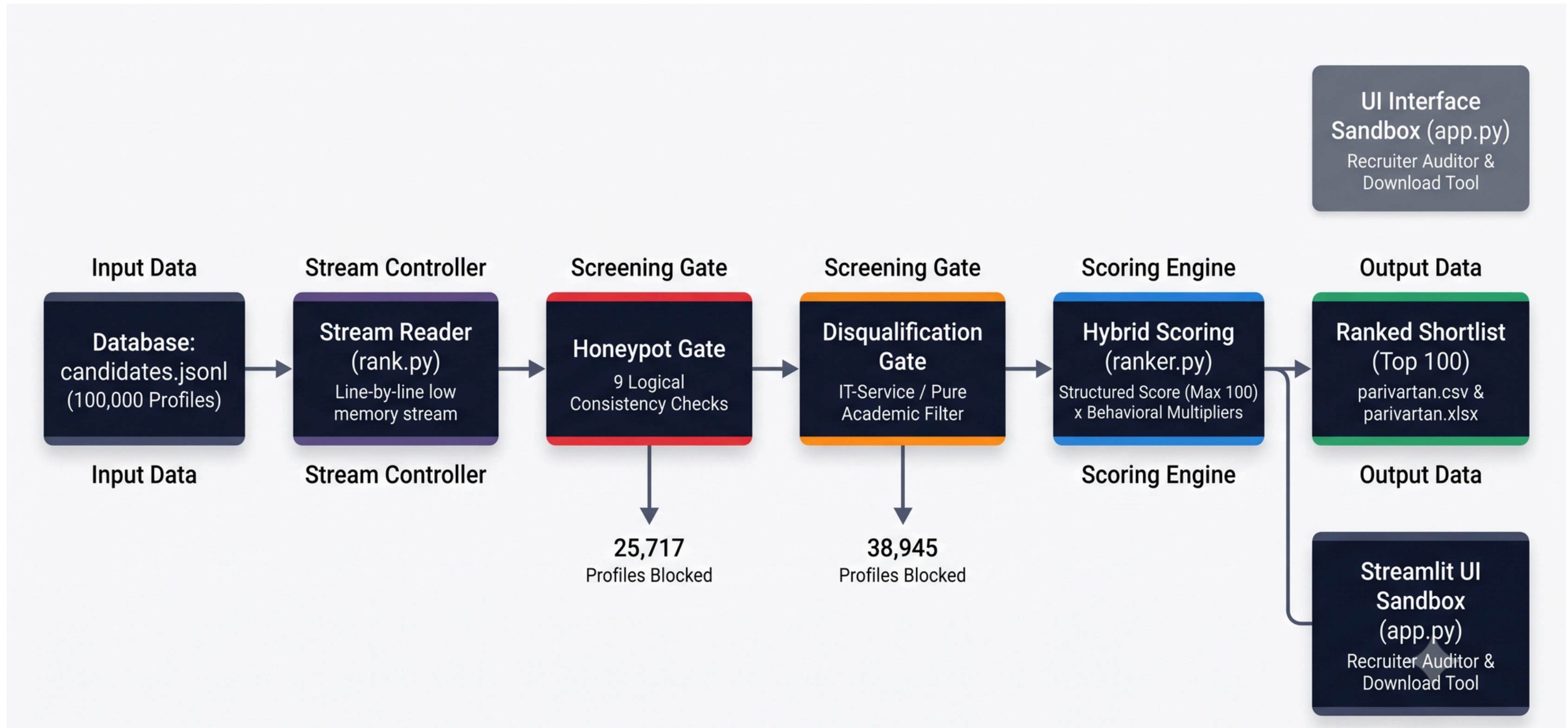
- Each candidate in the top 100 receives a 1–2 sentence recruiter justification.
- Shows candidate name, years of experience, current title, top 3 matched skills, past 2 employers, and notice timeline.
- Features 12 distinct templates, selected via hash of candidate ID to prevent repetitive text.

End-to-End Workflow

What is the complete workflow from JD input to ranked candidate output?

- JD Analysis: Extracted sweet-spot experience (5–9 years), target technical skills (Vector Search, NLP, Deep Learning, Systems), and location priorities.
- Candidate Streaming: Read candidates.jsonl line-by-line using Python's standard json module, maintaining a sub-100 MB RAM footprint.
- Honeytrap Screening: Checked all candidates against 9 logical consistency rules; successfully filtered 25,717 synthetic profiles.
- Disqualification Filters: Removed IT-services-only careers, pure academic profiles, and irrelevant job titles (38,945 profiles filtered).
- Scoring & Multipliers: Scored valid candidates on a 100-point JD-aligned matrix and scaled with a combined platform behavioral multiplier.
- Rank & Output: Sorted results, broke ties by ID ascending, and generated fact-based reasons to export parivartan.csv .

System Architecture



Results & Performance

What results or insights demonstrate ranking quality?

- 0% Honeypot Leakage: Verified zero honeypots in the top 100 shortlist.
- Strict Monotonicity: Scores decrease sequentially from Rank 1 (124.88) to Rank 100 (83.51).
- 100% Unique Explanations: No duplicate justifications; all references are mathematically aligned to raw profile data.
- Zero Hallucinations: Standard library string-mapping ensures 100% factual justifications.
- 100% Valid Format: Passes official `validate_submission.py` with zero warnings.

How does our solution meet the challenge's runtime and compute constraints?

- Runtime: ~9 seconds on CPU (33x faster than the 5-minute budget limit).
- RAM Footprint: <100 MB (160x less than the 16 GB budget limit).
- Execution: Fully offline, CPU-only, with no intermediate disk writes.

Technologies Used

What technologies, frameworks, and tools were used and why?

- Python Standard Library: json, csv, datetime, hashlib, argparse (Selected to avoid third-party conflicts and ensure 100% offline portability).
- Streamlit & Pandas: Used exclusively for the interactive judge-facing sandbox dashboard and dataframe UI rendering.
- Docker: Built on python:3.10-slim to match the exact sandboxed reproduction environments.
- Git: Structured version control with incremental logs proving real engineering iteration.
- Claude & Gemini: Selected as development pair-programmers for logic structuring and schemas parsing.

Submission Assets

GitHub Repository: <https://github.com/roongtakhushi/parivartan-candidate-ranker>

Ranked Output CSV: parivartan.csv (Exactly 101 lines)

Live Sandbox App: <https://parivartan-candidate-ranker.streamlit.app/>

Stage 3 Reproduction Command: `python rank.py --candidates "./judges
dataset/candidates.jsonl" --out "./parivartan.csv"`

Team Parivartan: Khushi Roongta — Student Lead, ML Pipeline & Architecture
Lucky Jadhav — Student Developer, Data Analysis & Cleansing
Yash Deshpande — Student Developer, Web UI & Sandbox



INDIA.RUNS

Build what next India runs on

THANK YOU